

Angular HTTP Interceptors

Angular 15+

1. Functional Interceptor (Recommended)

A plain function typed as **HttpInterceptorFn**. Introduced in Angular 15 — less boilerplate, tree-shakeable, and works natively with standalone apps.

interceptor file

```
// auth.interceptor.ts
import { HttpInterceptorFn } from '@angular/common/http';
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('access');
  if (token) {
    const cloned = req.clone({
      headers: req.headers.set('Authorization', `Bearer ${token}`),
    });
    return next(cloned);
  }
  return next(req);
};
```

app.config.ts — registration

```
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { authInterceptor } from './auth.interceptor';
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptors([authInterceptor]),
    ),
  ],
};
```

```
};
```

2. Class-Based Interceptor (Legacy)

Implements the **HttpInterceptor** interface. Still works in Angular 15+, but requires more boilerplate and a different registration method.

interceptor file

```
// auth.interceptor.ts
import { Injectable } from '@angular/core';
import { HttpHandler, HttpInterceptor, HttpRequest } from '@angular/common/http';
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest, next: HttpHandler) {
    const token = localStorage.getItem('access');
    if (token) {
      const cloned = req.clone({
        headers: req.headers.set('Authorization', `Bearer ${token}`),
      });
      return next.handle(cloned); // ← .handle(), not ()
    }
    return next.handle(req);
  }
}
```

app.config.ts — registration

```
import { HTTP_INTERCEPTORS } from '@angular/common/http';
import { provideHttpClient, withInterceptorsFromDi } from '@angular/common/http';
import { AuthInterceptor } from './auth.interceptor';
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptorsFromDi(), // ← required for class-based
    ),
    {
      provide: HTTP_INTERCEPTORS,
      useClass: AuthInterceptor,
      multi: true, // ← always true
    },
  ],
};
```

3. Quick Comparison

	Functional	Class-Based
Angular version	15+	Any
Boilerplate	Low	High
next() call	next(req)	next.handle(req)
Dependency inject.	inject()	Constructor
withInterceptors	withInterceptors([fn])	withInterceptorsFromDi()
Tree-shakeable	✓	✗
Recommended	✓	✗